

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Pruebas de Caja Negra: Partición de Equivalencia y Valores Límite				
NÚMERO DE PRÁCTICA:	05	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	18/05/2026	FECHA FIN:	22/05/2026	DURACIÓN:	100 min
RECURSOS A UTILIZAR: • Una computadora personal • Visual Studio Code / Python					
DOCENTE(s): • Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • Aplicar de forma sistemática la técnica de Partición de Equivalencia para reducir el número de casos de prueba manteniendo la efectividad. • Diseñar casos de prueba basados en Análisis de Valores Límite para detectar defectos en las fronteras de las especificaciones.	
TEMAS: • Pruebas de Caja Negra (especificación) • Clases de Equivalencia (válidas/inválidas) • Valores Límite	
COMPETENCIAS	C.m. Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustadas a los recursos disponibles del cliente

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

I. Introducción

En el desarrollo de software, garantizar la calidad de los sistemas es una tarea fundamental. Para ello existen diversas técnicas de prueba que permiten verificar el correcto funcionamiento de una aplicación antes de ser utilizada por los usuarios finales. Dentro de estas técnicas destacan las pruebas de tipo Black Box (Caja Negra), especialmente los métodos de Partición de Equivalencia y Análisis de Valores Límite, los cuales son ampliamente utilizados en pruebas funcionales.

Estas técnicas permiten diseñar casos de prueba eficientes, reducir el número de pruebas innecesarias y detectar errores relacionados con entradas inválidas, límites del sistema y validación de datos.

II. Fundamentos de las Pruebas de Caja Negra (Black Box Testing)

Las pruebas de caja negra, también conocidas como pruebas **basadas en la especificación** u orientadas a la entrada/salida, consideran al sistema como una entidad cuya estructura interna es desconocida o irrelevante para el probador.

- **Enfoque en el Comportamiento:** El objetivo principal es encontrar circunstancias en las que el programa no se comporte de acuerdo con sus especificaciones funcionales o no funcionales.
- **Puntos de Control y Observación:** Tanto el Punto de Control (PoC) (donde se envían los datos) como el Punto de Observación (PoO) (donde se verifican los resultados) se encuentran fuera del objeto de prueba.
- **Independencia del Código:** Los casos de prueba se derivan exclusivamente de la documentación de requisitos, modelos de uso o especificaciones externas, sin utilizar conocimiento del código fuente.
- **Filosofía Destructiva de Myers:** Las pruebas deben partir de la premisa de que el software contiene defectos; por lo tanto, una prueba se considera exitosa si detecta un error, no si demuestra que el programa funciona.

El enfoque de Black Box o Caja Negra consiste en evaluar el comportamiento externo de un sistema sin necesidad de conocer su estructura interna, código fuente o algoritmos.

El evaluador únicamente analiza:

- Las entradas proporcionadas al sistema
- Las salidas obtenidas
- El comportamiento esperado

El sistema es tratado como una “caja cerrada” donde no interesa cómo funciona internamente.

III. Partición de Equivalencia (PE)

La Partición de Equivalencia es una técnica de prueba que divide los datos de entrada en grupos llamados clases de equivalencia.

La idea principal es que:

- Todos los datos de una misma clase deberían comportarse de forma similar.
- Basta con probar un valor representativo de cada grupo.

Esto reduce la cantidad de pruebas necesarias.

Debido a que las pruebas exhaustivas (probar cada combinación posible de entrada) son prácticamente imposibles, la PE es una técnica sistemática para seleccionar un subconjunto de datos con alta probabilidad de detectar errores.

1. **Definición de Clase de Equivalencia:** Es un conjunto de datos de entrada para los cuales se asume razonablemente que el sistema mostrará un comportamiento idéntico.
2. **Representantes de Clase:** Probar un solo valor de una clase es suficiente para validar a todos los miembros de esa partición. Si un representante detecta un error, se asume que los demás también lo harían; si no lo hace, se asume que los demás tampoco.
3. **Tipos de Particiones:**
 - **Clases Válidas (vEC):** Representan entradas permitidas que el sistema debe procesar correctamente según los requisitos.
 - **Clases Inválidas (iEC):** Representan valores erróneos o inesperados. Probar estas clases es vital para validar la **robustez** y el manejo de excepciones del sistema.
4. **Reglas de Identificación:**
 - Si se especifica un **rango numérico** (ej. 1 a 999), se crea una clase válida y dos inválidas (menor que 1 y mayor que 999).
 - Si se requiere un **número de valores**, se crean clases para la cantidad correcta y particiones por defecto (menos o más de los requeridos).
 - Si hay un conjunto de valores ("debe ser A, B o C"), se identifica una clase válida por cada opción y una inválida para cualquier otra entrada.

IV. Análisis de Valores Límite (AVL)

El AVL es una extensión y complemento esencial de la Partición de Equivalencia. La experiencia demuestra que los defectos tienden a concentrarse en los **extremos o fronteras** de los rangos de entrada en lugar de en el "centro" de las clases.

- **Fronteras Críticas:** Los errores suelen ocurrir por interpretaciones erróneas de los requisitos (ej. confundir "menor que" con "menor o igual que") o por errores de programación tipo "off-by-one" (desfase por uno).
- **Selección de Datos:** A diferencia de la PE, donde se elige cualquier valor de la clase, el AVL exige probar valores directamente en el límite, justo por debajo y justo por encima de la frontera.
- **Criterio de los 2 o 3 valores:** En una frontera x , se deben probar idealmente:
 - El valor límite exacto (x).
 - El valor inmediatamente anterior ($x-1$).
 - El valor inmediatamente posterior ($x+1$).
- **Límites de Salida:** No solo se deben considerar las fronteras de las entradas, sino también diseñar casos que fuercen al sistema a generar resultados en sus límites máximos y mínimos de salida.

V. Sinergia y Estrategia de Aplicación

La combinación de PE y AVL permite maximizar el rendimiento de la inversión en pruebas al encontrar la mayor cantidad de defectos con el menor número de casos de prueba.

- **Desplazamiento a la Izquierda (Shift-Left):** Estas técnicas permiten diseñar pruebas desde la fase de requisitos, detectando ambigüedades en la especificación incluso antes de que se escriba una sola línea de código.
- **Métricas de Cobertura:** La suficiencia de la prueba se mide mediante la cobertura de clases de equivalencia y la cobertura de valores límite, calculando el porcentaje de particiones y fronteras ejercitadas frente al total identificado.
- **La Cadena de Calidad de Spillner:** Estas técnicas buscan prevenir que un Error (acción humana) se convierta en un Defecto (en el código) que finalmente resulte en una Falla (comportamiento observable incorrecto) para el usuario final.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Ejercicio Resuelto 1 – Partición de Equivalencia (PE)

Problema

Un sistema universitario permite registrar notas únicamente entre 0 y 20.

Crear casos de prueba usando la técnica de Partición de Equivalencia.

Paso 1: Identificar Clases de Equivalencia

Clase válida

- 0 – 20

Clases inválidas

- $x < 0$
- $x > 20$

Paso 2: Seleccionar Valores Representativos

Clase	Valor elegido	Resultado esperado
Válida	15	Aceptado
Inválida menor	-3	Rechazado
Inválida mayor	25	Rechazado

Paso 3: Creación de código y ejecución

```
def validar_nota(nota):  
    if 0 <= nota <= 20:  
        return "Nota válida"  
    else:  
        return "Nota inválida"
```

```
# Casos de prueba PE  
print(validar_nota(15))  
print(validar_nota(-3))  
print(validar_nota(25))
```

```
Nota válida  
Nota inválida  
Nota inválida
```

Ejercicio Resuelto 2 – Análisis de Valores Límite (AVL)**Problema**

Un sistema permite edades entre 18 y 60 años.

Aplicar la técnica de Valores Límite.

Paso 1: Identificar Límites

Límite inferior: 18

Límite superior: 60

Paso 2: Probar Valores Cercanos a los Límites

Entrada	Resultado esperado
17	Inválido
18	Válido
19	Válido
59	Válido
60	Válido
61	Inválido

Paso 3: Creación de código y ejecución

```
def validar_edad(edad):
    if 18 <= edad <= 60:
        return "Edad válida"
    else:
        return "Edad inválida"
```

```
Edad inválida
Edad válida
Edad válida
Edad válida
Edad válida
Edad inválida
```

Ejercicio Resuelto 3: VALIDADOR DE CATEGORÍA DE CRÉDITO

Escenario: Una función clasificar_cliente(score: int) que recibe un puntaje financiero (300 a 850) y retorna "Bajo", "Medio" o "Alto".

Paso 1: Análisis de Técnicas (PE y AVL de 3 Puntos)

Partición	Rango / Clase	Límite (Frontera)	Valor AVL	Tipo	Resultado Esperado
P1	< 300 (Inválida)	300	299	Frontera - 1	Value Error
P2	300 - 579 (Bajo)	300	300	Frontera	"Bajo"
		300	301	Frontera + 1	"Bajo"
		580	579	Frontera - 1	"Bajo"
P3	580 - 669 (Medio)	580	580	Frontera	"Medio"
		580	581	Frontera + 1	"Medio"
		670	669	Frontera - 1	"Medio"
P4	670 - 850 (Alto)	670	670	Frontera	"Alto"
		670	671	Frontera + 1	"Alto"
		850	849	Frontera - 1	"Alto"

		850	850	Frontera	"Alto"
P5	> 850 (Inválida)	850	851	Frontera + 1	Value Error

Paso 2: Código Fuente (credito.py)

```
def clasificar_cliente(score: int) -> str:
    """Clasifica un cliente según su score crediticio (300-850)."""
    if not isinstance(score, int):
        raise TypeError("El score debe ser un número entero.")

    if score < 300 or score > 850:
        raise ValueError("Score fuera de rango legal (300-850).")

    if score <= 579:
        return "Bajo"
    elif score <= 669:
        return "Medio"
    else:
        return "Alto"
```

Paso 3: Implementación de Pruebas (test_credito.py)

```
import pytest
from credito import clasificar_cliente

@pytest.mark.parametrize("score, esperado", [
    (300, "Bajo"), # Frontera inferior P2
    (301, "Bajo"), # Frontera + 1
    (579, "Bajo"), # Frontera - 1 (Transición a Medio)
    (580, "Medio"), # Frontera inferior P3
    (581, "Medio"), # Frontera + 1
    (669, "Medio"), # Frontera - 1 (Transición a Alto)
    (670, "Alto"), # Frontera inferior P4
    (671, "Alto"), # Frontera + 1
    (850, "Alto"), # Frontera superior P4
    (849, "Alto"), # Frontera - 1
])
def test_clasificar_cliente_validos(score: int, esperado: str):
    # Arrange: Preparar el escenario (datos de entrada ya parametrizados)

    # Act: Ejecutar la unidad bajo prueba
    resultado = clasificar_cliente(score)

    # Assert: Verificar el comportamiento esperado
    assert resultado == esperado

@pytest.mark.parametrize("val_invalido", [299, 851])
def test_clasificar_cliente_fuera_rango(val_invalido: int):
    # Act & Assert: Verificar que se lanza ValueError en límites inválidos
    with pytest.raises(ValueError):
        clasificar_cliente(val_invalido)

def test_clasificar_cliente_tipo_erroneo():
    # Act & Assert: Verificar blindaje contra tipos no enteros (Robustez)
    with pytest.raises(TypeError):
        clasificar_cliente("800")
```

Paso 4: Verificación

```
test_credito.py::test_clasificar_cliente_validos[300-Bajo] PASSED [ 7%]
test_credito.py::test_clasificar_cliente_validos[301-Bajo] PASSED [ 15%]
test_credito.py::test_clasificar_cliente_validos[579-Bajo] PASSED [ 23%]
test_credito.py::test_clasificar_cliente_validos[580-Medio] PASSED [ 30%]
test_credito.py::test_clasificar_cliente_validos[581-Medio] PASSED [ 38%]
test_credito.py::test_clasificar_cliente_validos[669-Medio] PASSED [ 46%]
test_credito.py::test_clasificar_cliente_validos[670-Alto] PASSED [ 53%]
test_credito.py::test_clasificar_cliente_validos[671-Alto] PASSED [ 61%]
test_credito.py::test_clasificar_cliente_validos[850-Alto] PASSED [ 69%]
test_credito.py::test_clasificar_cliente_validos[849-Alto] PASSED [ 76%]
test_credito.py::test_clasificar_cliente_fuera_rango[299] PASSED [ 84%]
test_credito.py::test_clasificar_cliente_fuera_rango[851] PASSED [ 92%]
test_credito.py::test_clasificar_cliente_tipo_erroneo PASSED [100%]
```


III. EJERCICIOS/PROBLEMAS PROPUESTOS

EJERCICIO PROPUESTO: Guerra de Testers

La actividad “Guerra de Testers” consiste en una competencia académica donde los estudiantes trabajarán en equipos para:

1. Diseñar y programar una aplicación.
2. Aplicar pruebas Black Box:
 - Partición de Equivalencia (PE)
 - Análisis de Valores Límite (AVL)
3. Intercambiar aplicaciones con otro equipo.
4. Intentar encontrar errores, fallos o validaciones incorrectas en el software rival.

La dinámica busca desarrollar habilidades de:

- Programación
- Testing
- Validación de software
- Detección de errores
- Documentación técnica
- Pensamiento crítico

Dinámica General

Fase 1 – Desarrollo

Cada grupo seleccionará una aplicación de una lista propuesta por el docente.

Cada equipo tendrá hasta 03 días para desarrollar el sistema (DEV) y 03 días para probar el sistema (QA) del otro equipo, los equipos pueden hacer uso de asistentes de código: Gemini Code Assist, Codex de OpenAI o Claude Code, deberían incluir en el informe los prompts usados para generar las aplicaciones.

Fase 2 – Testing Interno

Antes de entregar su aplicación, cada equipo deberá:

Aplicar enfoque Black Box:

- Partición de Equivalencia (PE)
- Valores Límite (AVL)

Entregar:

- Repositorio con el código fuente o archivo comprimido .zip
- Casos de prueba (pruebas unitarias)
- Resultados esperados
- Evidencia de validaciones

Fase 3 – Guerra de Testers

Los grupos intercambiarán aplicaciones.

Cada equipo actuará como:

QA Tester / Equipo atacante

Su misión será:

- Encontrar errores.
- Romper validaciones.
- Detectar comportamientos inesperados.
- Probar entradas inválidas.
- Buscar fallos límite.

Reglas de la Competencia**Permitido**

- Probar cualquier entrada válida o inválida.
- Buscar errores lógicos.
- Intentar romper formularios.
- Evaluar límites extremos.
- Detectar excepciones.

No permitido

- Modificar código fuente rival.
- Eliminar archivos.
- Ataques externos reales.
- Uso de malware.
- Cambiar configuraciones del sistema.

Requisitos Obligatorios del Software

Cada aplicación debe incluir:

Validaciones

Ejemplos:

- Edad válida
- Contraseña mínima
- Límites de montos
- Campos obligatorios

Restricciones

Ejemplo:

$0 \leq \text{nota} \leq 20$

Manejo de Errores

El sistema no debe cerrarse inesperadamente.

Parte Obligatoria de Testing

Cada grupo deberá crear:

1. Casos PE (Partición de Equivalencia)

Ejemplo:

Clase	Valor	Resultado
Válida	15	Correcto
Inválida	-2	Error

2. Casos AVL (Valores Límite)

Ejemplo:

Entrada	Resultado
0	Correcto
20	Correcto
21	Error

Entregables

Cada grupo debe entregar:

1. Código fuente

Puede ser en:

- Python
- Java
- C#
- JavaScript

2. Documento técnico

Debe incluir:

- Nombre del sistema
- Integrantes
- Descripción
- Validaciones
- Casos PE
- Casos AVL

3. Manual rápido de uso

Explicar:

- Cómo ejecutar el programa.
- Datos válidos.
- Restricciones.

4. Documento de búsqueda de errores

Cada equipo deberá entregar un documento donde registre los errores encontrados en la aplicación del grupo rival.

El documento debe incluir:

- Nombre del equipo evaluado.
- Nombre del sistema evaluado.
- Descripción del error encontrado.
- Tipo de error detectado.
- Evidencia del error (capturas o explicación).
- Pasos para reproducir el error.
- Resultado esperado.
- Resultado obtenido.
- Nivel de gravedad del bug.

Nº	Aplicación
1	Sistema de Gestión de Gimnasio y Nutrición
2	Plataforma de Venta de Entradas para Eventos
3	Sistema de Restaurante y Delivery
4	Plataforma de Cursos Online con Gamificación
5	Sistema de Biblioteca Inteligente
6	Sistema Bancario Personal
7	Sistema de Clínica Veterinaria
8	Plataforma de Gestión de Torneos Deportivos
9	Sistema de Inventario y Ventas
10	Plataforma de Reservas de Citas Médicas
11	Sistema de Gestión de Hotel
12	Videojuego
13	Sistema de Gestión de Taller Mecánico
14	Aplicación de Seguimiento de Hábitos

15	Plataforma de Gestión de Tareas Colaborativas
16	Sistema de Registro y Control de Asistencia
17	Plataforma de Subastas Online
18	Sistema de Gestión de Cine
19	Aplicación de Control Financiero Personal
20	Sistema de Administración de Condominios

I. CUESTIONARIO

1. ¿Por qué es importante considerar los valores de salida al realizar un análisis de valores límite?
2. Explique cómo la competencia de intercambio de código ayuda a mitigar la "Ceguera ante los propios errores" descrita por Myers.
3. ¿Qué es el "Pesticide Paradox" y por qué el diseño sistemático (PE/AVL) ayuda a combatirla?
4. Según Myers, ¿por qué una prueba que no encuentra errores se considera "no exitosa"?
5. Defina la diferencia entre clase de equivalencia inválida y valor de entrada inválido

II. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

[1] Spillner, A., *Software Testing Foundations*, 5th Ed., 2021.

[2] Myers, G., *The Art of Software Testing*, 3rd Ed., 2012.

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

*Ejercicios propuestos /
Cuestionario*

INSTRUMENTOS:

Rúbrica

CRITERIOS DE EVALUACIÓN

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (55%)	Insuficiente (30%)	No presenta (0%)
Claridad y completitud de la explicación	Todos los ejercicios propuestos han	Los ejercicios propuestos han sido resueltos y	Los ejercicios propuestos han sido resueltos,	Los ejercicios han sido resueltos	No presentan los ejercicios resueltos, o la

de los ejercicios propuestos. (30 %)	sido explicados de forma muy clara y con la profundidad adecuada.	explicados, pero de manera muy superficial o general.	sin embargo, algunos de ellos no han sido explicados.	parcialmente y no tienen explicaciones de lo realizado.	versión final del informe omite por completo esta sección.
Ejercicios Propuestos (Código fuente) (20 %)	Todos los ejercicios propuestos se ejecutan correctamente y devuelven los resultados esperados.	Los ejercicios propuestos han sido implementados pero con los resultados esperados parciales.	Por lo menos el 2/3 de los ejercicios propuestos han sido implementados con resultados	Por lo menos el 1/3 de los ejercicios propuestos han sido implementados con resultados	No se han implementado los ejercicios propuestos.
Cuestionario (30%)	Todas las respuestas son completas, correctas y proporcionan explicaciones claras, incluyen citas.	La mayoría de las respuestas son correctas, pero algunas carecen de la profundidad esperada, pocas citas.	Algunas respuestas son correctas, pero incompletas o con falta de claridad en las explicaciones, citas no relevantes.	La mayoría de las respuestas son escuetas y/o incompletas, no hay evidencia de haber buscado información que las respalden, sin citas.	No presenta el cuestionario.
Presentación del Informe (20%)	El informe está bien estructurado, con excelente redacción, sin errores	El informe está bien estructurado, con algunos errores menores de redacción	El informe presenta problemas en la organización o redacción, con varios errores	El informe está desorganizado, con múltiples errores de redacción y formato	No presenta el informe.